<TP2>

Trabajo Práctico Grupal 2

Proyecto React en equipo

Construyan una aplicación que presente al equipo, organice sus contenidos y tenga una identidad propia. Usen React y la asistencia de IA para desarrollar, probar y personalizar el proyecto.

Modalidad: grupal

Fechas: se comunicarán por Moodle

Entrega: un enlace al repositorio de TP2

01 Organizar, construir y apropiarse

El TP continúa el proyecto del equipo iniciado en TP1 y lo lleva a React. El objetivo es organizar el trabajo, construir una aplicación navegable y tomar decisiones sobre su contenido y su diseño.

La sidebar organiza la experiencia

El menú lateral será una estructura compartida: permanece como referencia mientras cambia el contenido de cada sección. Les permite trabajar componentes reutilizables y organizar la navegación de la aplicación.
La IA forma parte del proceso. Pueden usarla para programar, revisar errores, explorar diseños y generar o editar imágenes. El equipo elige qué incorporar, lo adapta a su propuesta y comprueba que funcione.
La relación con TP1 se muestra en la nueva aplicación y en su bitácora. No se pide un informe comparativo ni capturas de antes y después.

1. **Organicen.** Creen el repositorio, incorporen a todo el equipo y completen la información básica del README.
2. **Construyan.** Armen la estructura React, la sidebar y las secciones que se describen abajo.
3. **Personalicen y prueben.** Revisen perfiles, contenidos, imágenes, navegación y datos.
4. **Publiquen y entreguen.** Comprueben los enlaces y recorran los 15 criterios antes de entregar.

02 Qué tienen que construir

Los requisitos y la rúbrica tienen la misma numeración. Cada punto indica qué debe estar presente; la rúbrica explica los niveles de logro.

01 Repositorio y publicación

Creen un repositorio público independiente para TP2 y publiquen la aplicación en Vercel. Comprueben ambos enlaces sin iniciar sesión.

02 README obligatorio

Completen el README.md en la raíz: descripción del proyecto, enlace al deploy, nombres de todos los integrantes y enlaces a sus perfiles de GitHub.

03 Integrantes con acceso al repositorio

Incorporen a todos los integrantes registrados del grupo al repositorio. Cada persona invitada debe aceptar la invitación. Quien creó el repositorio ya cuenta como integrante con acceso.

04 Proyecto desarrollado en React

Desarrollen la aplicación en React, con sus pantallas construidas mediante componentes y navegación mediante React Router. Continúen el proyecto del equipo iniciado en TP1.

05 Sidebar compartida

Organicen la aplicación desde una sidebar: un menú lateral compartido que muestre la identidad del equipo, permita acceder a las secciones e indique cuál está activa.

06 Navegación completa

Todas las pantallas deben permitir continuar o volver mediante la sidebar, enlaces o botones de la aplicación. Nadie debe depender del botón Atrás del navegador.

07 Portada del equipo

Diseñen una portada que presente al equipo con nombre, descripción y acceso a sus integrantes. La portada debe tener una presentación propia, además de las tarjetas o enlaces a los perfiles.

08 Perfiles dentro de React

Construyan un perfil por integrante dentro de esta aplicación React. Cada perfil debe identificar a la persona y presentar su contenido. Los enlaces a las páginas del TP1 no reemplazan los perfiles del TP2.

09 Apropiación y propuesta estética

Elijan una identidad visual para el equipo: colores, tipografías, composición e imágenes. Pueden usar IA para explorar estilos y generar o editar avatares, fotos, ilustraciones o fondos. Revisen el resultado y personalícenlo para su proyecto; no es obligatorio usar fotos personales.

10 Datos locales JSON

Incluyan un archivo JSON local con al menos 20 registros relacionados con la propuesta del equipo y muéstrenlos dinámicamente en tarjetas o en un listado. Por ejemplo: proyectos, recursos o herramientas.

11 Búsqueda y filtro

Sobre el listado del JSON, incorporen una búsqueda por texto y un filtro por una propiedad de los datos. Ambos deben actualizar los resultados en la aplicación.

12 API pública y estados

Elijan una API pública, indiquen en la sección qué información aporta y muestren sus resultados. Implementen un mensaje mientras carga y otro ante un error. Elijan una API que puedan consultar desde el navegador sin exponer claves privadas.

13 Árbol de renderizado

Incluyan una sección con el árbol de componentes de su aplicación: el componente principal y sus hijos, como layout, sidebar, páginas y tarjetas. Debe corresponder al código entregado. No es sólo una lista de carpetas.

14 Bitácora

Incluyan una sección con registros breves del proceso de trabajo: qué hicieron, qué decidieron o qué dificultad resolvieron. Pueden continuar y actualizar la bitácora del TP1 dentro de la nueva aplicación.

15 Declaración de uso de IA

Registren en el README las aplicaciones y los modelos de IA utilizados. Diferencien la aplicación desde la que trabajaron del modelo empleado. Para ampliar la declaración, indiquen qué usó cada integrante y en qué contenido o tarea.

Una aplicación conectada: portada, perfiles, datos locales, API, árbol de renderizado y bitácora deben ser accesibles desde la navegación interna. El proyecto se presenta y se recorre dentro de React.

03 README y entrega

Un README breve y completo

Descripción: qué propone el proyecto y qué contiene.

Deploy: enlace funcional al sitio publicado en Vercel.

Integrantes: nombre y apellido de cada persona, con enlace a su perfil de GitHub.

Uso de IA: aplicación y modelo utilizados. Este apartado corresponde al criterio 15.

Para los niveles **Propone**, pueden sumar índice y capturas, responsabilidades del equipo y detalle del uso de IA por integrante. Una plantilla con campos vacíos no cuenta como documentación completa.

Un único enlace de entrega

Carguen el enlace al **repositorio público de TP2** en la pestaña **TP 2** de la planilla única de entregas, en la fila de su grupo. El enlace a Vercel debe estar dentro del README.

Antes de entregar, abran el repositorio y el sitio sin iniciar sesión. Revisen todos los perfiles y secciones, la búsqueda, los filtros y la API. Confirmen que todas las invitaciones al repositorio estén aceptadas.

04 Rúbrica de evaluación

Se selecciona un nivel por criterio. Los descriptores explican qué corresponde a cada resultado.
1 · No cumple / 2 · Cumple / 3 · Propone. Propone incluye todo lo exigido en Cumple. El criterio 9 agrega **4 · Supera**, que incluye los niveles anteriores.

01 · Repositorio y publicación

1. No cumple: El repositorio no es público o el sitio en Vercel no está accesible. Basta que falle uno.

2. Cumple: Repositorio público y sitio accesible en Vercel.

3. Propone: Además, la presentación del repositorio incluye descripción del proyecto y acceso directo al sitio.

02 · README obligatorio

1. No cumple: Falta alguno: descripción, enlace al deploy, nombres de todos los integrantes o enlaces a sus perfiles de GitHub.

2. Cumple: Incluye los cuatro elementos completos y sus enlaces funcionan.

3. Propone: Además, incluye índice con enlaces internos y capturas del proyecto.

03 · Integrantes con acceso al repositorio

1. No cumple: Falta al menos un integrante con acceso aceptado al repositorio.

2. Cumple: Todos los integrantes tienen acceso, contando al propietario del repositorio.

3. Propone: Además, el README identifica la responsabilidad de cada integrante.

04 · Proyecto desarrollado en React

1. No cumple: La aplicación no está desarrollada en React. Instalar la dependencia sin utilizarla no alcanza.

2. Cumple: La aplicación utiliza React y renderiza sus pantallas mediante componentes.

3. Propone: Además, reutiliza un mismo componente con distintos datos mediante props.

05 · Sidebar compartida

1. No cumple: Falta la sidebar o no incluye identidad del equipo, enlaces funcionales y ubicación actual.

2. Cumple: Sidebar compartida entre las secciones, con identidad, enlaces y sección activa identificada.

3. Propone: Además, se adapta a móvil mediante un panel que abre y cierra correctamente.

06 · Navegación completa

1. No cumple: Alguna pantalla queda sin salida interna o exige usar Atrás del navegador.

2. Cumple: Todas las pantallas permiten continuar o volver mediante controles de la aplicación.

3. Propone: Además, se puede abrir y recargar directamente la URL de cada sección sin errores.

07 · Portada del equipo

1. No cumple: La portada es únicamente un listado o conjunto de perfiles.

2. Cumple: Tiene una presentación del equipo con nombre, descripción y acceso a los perfiles.

3. Propone: Presentación visual cuidada con elementos interactivos que introducen al equipo y sus perfiles.

08 · Perfiles dentro de React

1. No cumple: Falta un perfil, alguno no funciona o enlaza a una página del trabajo anterior.

2. Cumple: Todos los integrantes tienen un perfil funcional dentro de la aplicación React.

3. Propone: Además, todos los perfiles incluyen una interacción funcional sobre su contenido.

---

09 · Apropiación y propuesta estética

1. No cumple: No hay estilos aplicados o la presentación impide leer o utilizar el sitio.

2. Cumple: Presentación funcional con estética genérica, incluida una propuesta de IA utilizada sin personalización visible.

3. Propone: Paleta, tipografías, composición y recursos visuales muestran una identidad consistente del equipo.

4. Supera: Propuesta estética integral del equipo, con imágenes generadas o editadas para el proyecto, composición personalizada y coherencia visual entre portada, perfiles y demás secciones.

---

10 · Datos locales JSON

1. No cumple: No hay JSON local, tiene menos de 20 registros o sus datos no se muestran dinámicamente.

2. Cumple: JSON local con al menos 20 registros relacionados con el proyecto y mostrados dinámicamente.

3. Propone: Además, cada registro permite desplegar o consultar información adicional.

---

11 · Búsqueda y filtro

1. No cumple: Falta la búsqueda o el filtro, o alguno no modifica correctamente los resultados.

2. Cumple: Búsqueda por texto y filtro funcionales sobre los datos locales.

3. Propone: Además, informa cuando no hay resultados y permite restablecer la lista completa.

---

12 · API pública y estados

1. No cumple: No consume una API pública, no muestra sus resultados o falta implementar carga o error.

2. Cumple: Consume una API pública, muestra resultados e implementa estados de carga y error.

3. Propone: Además, permite actualizar o reintentar la consulta desde la interfaz.

---

13 · Árbol de renderizado

1. No cumple: No está o no corresponde a los componentes del proyecto.

2. Cumple: Imagen o esquema estático que representa la jerarquía real de componentes.

3. Propone: El árbol permite expandir y contraer sus componentes.

---

14 · Bitácora

1. No cumple: No está o está vacía.

2. Cumple: Sección estática con registros del proceso de trabajo del equipo.

3. Propone: Los registros se presentan mediante desplegables o filtros funcionales.

---

15 · Declaración de uso de IA

1. No cumple: No declara IA, nombra sólo la aplicación o nombra sólo el modelo.

2. Cumple: Identifica aplicación y modelo utilizados.

3. Propone: Identifica, por integrante, aplicación, modelo y contenidos o tareas para los que se utilizaron.
